

COGNOMS:

[illegible]

NOM:

[illegible]

Problema 2. (3.6 puntos)

NOTAS: Las preguntas teóricas sin justificación, con justificación incompleta o incorrecta se consideraran erróneas. Considera el caso más probable; ejemplo: *Una cache asociativa tiene menos fallos que una directa, pero hay casos en los que es al revés.*

Un fabricante de teléfonos móviles está desarrollando su próximo teléfono de gama alta. El principal componente es un SOC (System on a Chip) que integra, entre otros, varias CPUs de propósito general, varios DSPs con arquitectura VLIW, una GPU, etc. Para reducir el consumo (y por tanto extender la duración de la batería) durante la grabación de vídeo, el software de procesamiento de vídeo se ejecuta sobre uno de los DSP VLIW. Se ha observado que una parte importante del consumo del procesamiento de vídeo se debe a la jerarquía de memoria y en particular a los accesos a datos.

El procesador VLIW dispone de una cache de datos de mapeo directo con política de escritura **Write Through + Write No Allocate**. Hemos estimado que un acceso a memoria principal consume del orden de 10 veces mas que un acceso a la cache de datos. El fabricante del SOC nos ofrece la posibilidad de mejorar el consumo de los accesos a datos a base de añadir un buffer de escritura con dos posibles variantes: un buffer convencional de ancho 1 palabra o bien un merge buffer. En ambos casos el consumo por los accesos al buffer es bastante mas pequeño (aproximadamente 1/4) que los accesos a la cache, pero no es despreciable.

- a) **Indica** para cada una de las mejoras si el consumo aumentará (A), se quedará igual (I) o se reducirá (R). **Justifica** la respuesta.

Mejora	Consumo	Justificación
Añadir un buffer de escrituras	A	El número de escrituras a memoria principal es el mismo y hemos añadido el consumo del buffer. (ver transparencia 38 de 050 cache avanzada)
Añadir un merge buffer (de escrituras)	R	El merge buffer reduce el numero de escrituras a memoria principal al combinar varias escrituras en un único acceso. Lo que compensa el consumo del buffer de escritura. (ver transparencia 39 de 050 cache avanzada)

Finalmente el fabricante del SOC ha decidido desechar la incorporación del buffer de escritura y cambiar la política de escritura a **Copy back + Write Allocate**. Sabemos que la tasa de fallos es de un 10% con ambas políticas de escritura y que las escrituras representan el 30% de los accesos a datos.

- b) **Indica** si el consumo aumentará (A), se que quedará igual (I) o se reducirá (R), con el cambio de política de escritura. **Justifica** la respuesta.

Mejora	Consumo	Justificación
Cambiar la política de escritura a Copy back + Write Allocate	R	Con WT tenemos 37 accesos a MP por cada 100 de datos (30 de escritura y 7 de fallos de lectura) Con CB tenemos 10 accesos por fallos de cache + los accesos por copy back, que aunque no tenemos datos estarían en el rango 0-10.

El fabricante del SOC nos ofrece cuatro posibles mejoras para la cache directa con **Copy back + Write Allocate**.

- c) **Indica** para cada una de ellas si el consumo aumentará (A), se que quedará igual (I) o se reducirá (R). **Justifica** la respuesta (continúa en la otra cara).

Mejora	Consumo	Justificación
Predictor de vía	A	No tiene sentido en una cache de mapeo directo. El consumo aumentara levemente por el predictor.
En caso de DB = 1: Actualizar MP después de leer la línea	I	No cambia el numero de accesos a MP, solo el orden. (ver transparencia 32 de 050 cache avanzada)

Mejora	Consumo	Justificación
Continuación anticipada (Early Restart)	I	No cambia el numero de accesos a MP, solo el momento que la CPU recibe el dato. (ver transparencia 33 de 050 cache avanzada)
Añadir prefetch hardware	A	El prefetch reduce los fallos, pero no los accesos a MP. El prefetch genera accesos innecesarios a MP, por lo que el consumo aumentará.

Al mismo tiempo, los ingenieros de computadores han analizado el código del procesado de vídeo y han descubierto algunos fragmentos de código ejecutados muy frecuentemente que se podrían optimizar para reducir el consumo.

Sabemos que en el siguiente fragmento la constante **M** es múltiplo de 1024, la cache directa tiene un tamaño de 4Kbytes y el tamaño de bloque es de 32 bytes.

```
float A[M], B[M];

for (i=0; i<M; i++)
    A[i] = A[i] + B[i] * 2.57;
```

d) **Realiza** una optimización de código de las vistas en clase que nos ayude a reducir el consumo.

Podríamos hacer padding para eliminar los conflictos y aprovechar la localidad espacial. Hay que dejar al menos un bloque de cache de relleno para eliminar totalmente los conflictos.

```
float A[M], p[8], B[M];
```

Sabemos que en el siguiente fragmento la constante **N** es muchísimo mayor que el tamaño de cache y no hay conflictos entre las variables **a,b,c,d**.

```
for(i=0; i < N; i++)
    a[i] = b[i] * c[i];
for(i=0; i < N; i++)
    d[i] = a[i] * c[i];
```

e) **Realiza** una optimización de código de las vistas en clase que nos ayude a reducir el consumo.

Podríamos fusionar los bucles para aprovechar la localidad temporal.

```
for(i=0; i < N; i++) {
    a[i] = b[i] * c[i];
    d[i] = a[i] * c[i];
}
```

Sabemos que en el siguiente fragmento la constante **N** es mayor que el tamaño de cache y no hay conflictos entre las variables **ma,mb,mc**.

```
for(i=1; i < N-1; i++)
    for(j=0; j < N; j++)
        ma[j][i] = (mb[j][i+1] + mc[j][i-1])*0.5;
```

f) **Realiza** una optimización de código de las vistas en clase que nos ayude a reducir el consumo.

Podríamos hacer intercambio de bucles para aprovechar la localidad espacial.

```
for(j=0; j < N; j++)
    for(i=1; i < N-1; i++)
        ma[j][i] = (mb[j][i+1] + mc[j][i-1])*0.5;
```

[illegible][illegible]

Problema 3. (1.4 puntos)

Nuestro equipo doméstico dispone de un disco duro cuyo coste es de 96 euros, del que desconocemos su capacidad de almacenamiento. Tras ejecutar un programa de chequeo hemos obtenido la siguiente información sobre el disco:

platos / caras por plato	8 / 2
pistas	610352 pistas/carra
tamaño sector	512 bytes
seek time medio (situar el cabezal en el cilindro)	12 ms
latencia media (tiempo que tarda en pasar el sector deseado)	2 ms
transfer rate (ancho de banda durante la transferencia de datos)	30 MB/s

- a) **Calcula** el coste por GB del disco si éste se formatea con una densidad fija de 64 sectores por pista. Escribe el resultado en GB (potencias de 10).

$$\text{Cdisco} = n_{\text{platos}} \times n_{\text{caras}} \times n_{\text{pistas}} \times n_{\text{sectores}} \times \text{tam_sector} = 8 \times 2 \times 610352 \times 64 \times 512 \text{ B} = 320 \text{ GB}$$

$$\text{Coste} = 96 \text{ euros} / 320 \text{ GB} = 0,30 \text{ euros/GB}$$

Una aplicación P realiza lecturas de bloques de datos de 30 KBytes cuyos sectores se encuentran consecutivos en la misma pista, por lo que el tiempo de búsqueda y el de posicionamiento (seek + latencia) solo se pierde una vez por cada bloque (y no por cada uno de los sectores que lo componen). Supondremos que la velocidad de rotación es la adecuada para que el disco sea capaz de realizar la transferencia a la velocidad del bus, y consideraremos despreciables los tiempos dedicados al procesamiento de las peticiones, cálculo de CRCs, DMA, etc.)

- b) **Calcula** el tiempo total necesario para leer un bloque de 30 KBytes, y el ancho de banda efectivo al leer un bloque de información de 30 KBytes de este disco (escribe el resultado en MB/s).

$$T_{\text{transf}} = 30 \text{ KB} / 30 \text{ MB/s} = 1 \text{ ms}$$

Tiempo = seek + latencia + Ttransf = 12 + 2 + 1 = 15 ms (ver Problema 1 del Tema 4)

$$AB = 30 \text{ Kbytes} / 15 \text{ ms} = 2 \text{ MBytes/s}$$

Para mejorar el acceso a los datos de la aplicación P necesitaremos un sistema de almacenamiento que nos dé mayor ancho de banda que el actual, además de añadir algún mecanismo de tolerancia a fallos en disco. Valoraremos la compra de una serie de discos duros nuevos de 300 GBytes cada uno, y la organización de los sistemas RAID 10 (con mirror doble), RAID 5 y RAID 6. La aplicación P necesita un total de 3,6 TBytes de almacenamiento.

- c) **Calcula** cuántos discos duros de 300 GBytes necesitaríamos para cada uno de los niveles RAID 10 (mirrор doble), 5 y 6 si queremos almacenar al menos 3,6 TBytes útiles.

RAID 10: 3600 GB x 2 mirrors / 300 GB/d = 24 discos

RAID 5: 3600 GB / 300 GB/d + 1d paridad = 13 discos

RAID 6: 3600 GB / 300 GB/d + 2d paridad = 14 discos

El sistema de disco se ha organizado como un RAID 0 con 12 discos. Cada disco tiene un tiempo medio entre fallos ($MTTF_{\text{disco}}$) de 100000 horas, y existen suficientes peticiones de acceso para saturar el ancho de banda de los discos. Disponemos de un backup total en cinta con un ancho de banda efectivo de 50 MBytes/s. Si falla un disco del RAID podemos reemplazarlo y recuperar una copia completa de los datos desde el backup. Suponer que la sustitución del disco fallido por uno nuevo es inmediata y se pueden empezar a recuperar los datos tan pronto se detecta el fallo.

- d) **Calcula** el MTTF en horas y la disponibilidad del sistema RAID 0 con 12 discos.

$$MTTF_{RAID0} = 100000 \text{ h/d} / 12 \text{ discos} = 8333 \text{ horas}$$

Trecuperarbackup = MTTR = $(3,6 \times 10^{12} \text{ bytes} / 50 \times 10^6 \text{ bytes/s}) / 3600 \text{ s/h} = 20 \text{ horas}$

Disponibilidad = $MTTF / (MTTF + MTTR) = 8333 \text{ h} / 8353 \text{ h} = 0,99$ (ver Problema 12 del Tema 1)

[illegible][illegible]

Problema 4. (3,6 puntos).

Disponemos de un procesador CISC en el que las instrucciones x86 se traducen internamente a microoperaciones (uops). A continuación se presenta un código ensamblador del x86 y su traducción a la correspondiente secuencia de uops. Las variables **V** y **x** son de tipo entero (4 bytes), y el valor de la constante **N** por ahora es desconocido:

código x86: <pre> movl \$0, %ecx loop: cmpl \$N, %ecx jge fin movl x, %eax imull V(,%ecx,4), %eax movl %eax, V(,%ecx,4) incl %ecx jmp loop fin: </pre>	código uops: <pre> movl %ecx <- \$0 loop: cmpl %ecx, \$N jge fin load %eax <- M[x] load %r1 <- M[%ecx*4 + V] imull %eax <- %r1 * %eax store M[%ecx*4 + V] <- %eax addl %ecx <- %ecx + \$1 jmp loop fin: </pre>
--	--

Suponer que las uops ocupan todas 4 bytes, y que las instrucciones x86 ocupan:

- 1 byte de código de operación (OpCode)
- 1 byte adicional si tiene 2 o más operandos (Modo), las de 1 sólo operando lo codifican dentro del OpCode
- 2 bytes adicionales si alguno de los operandos es un Inmediato (incluidos los destinos de los saltos)
- 1 byte adicional (SIB) si accede a memoria
- 2 bytes adicionales si el modo de direccionamiento incluye Desplazamiento

- a) **Calcula** cuantas instrucciones estáticas y dinámicas se ejecutan y cuantas uops estáticas y dinámicas se ejecutan. Da el resultado en función de **N**. **Calcula** también el tamaño en bytes del código x86 y el que ocuparían las uops equivalentes. Al calcular el tamaño del código x86 debes indicar claramente los bytes que ocupa cada instrucción.

Estáticas: 8 instrucciones, 9 uops

Dinámicas: $7 \times N$ (+3) instrucciones, $8 \times N$ (+3) uops

Tamaño del código x86 = 4 (movl) + 4 (cmpl) + 3 (jge) + 5 (movl) + 5 (imull) + 5 (movl) + 1 (incl) + 3 (jmp) = 30 bytes

Tamaño del código uops = $9 \times 4 = 36$ bytes

(ver Problema 3 del Tema 5)

Sabemos ahora que en el código anterior la constante **N** vale 10^9 , y que el código se ejecuta en 6,4 segundos y con un CPI de 1,5 ciclos/instrucción.

- b) **Calcula** cual es el UPC (uops por ciclo) que se obtiene al ejecutar el código. **Calcula** también la frecuencia a la que funciona el procesador.

$$\text{ciclos} = 7 \times 10^9 \text{ instr} * 1,5 \text{ c/i} = 10,5 \times 10^9 \text{ ciclos}$$
$$\text{UPC} = 8 \times 10^9 \text{ uops} / 10,5 \times 10^9 \text{ ciclos} = 0,76 \text{ uops/c}$$
$$F = 10,5 \times 10^9 \text{ ciclos} / 6,4 \text{ seg} = 1,6 \text{ GHz}$$

(ver Problema 3 del Tema 5)

El procesador dispone de una cache de instrucciones de nivel 1 (L1) y de una cache de microoperaciones de nivel 0 (L0) donde se guardan las uops que ya han sido traducidas y que permite evitar traducir repetidas veces las mismas instrucciones del código x86.

- c) **Calcula** el ancho de banda efectivo de la cache de instrucciones al ejecutar el código sin uop cache, y el ancho de banda efectivo de la cache de uops al ejecutar el código con uop cache. Escribe los dos resultados en GBytes/s.

$$\text{Bytes} = (4 + (4+3+5+5+5+1+3) \times 10^9 + 4+3) \text{ bytes} = (26 \times 10^9 + 11) \text{ bytes}$$

$$\text{AB-I} = 26 \times 10^9 \text{ bytes} / 6,4 \text{ s} = 4 \text{ Gbytes/s}$$

$$\text{Bytes} = (4 + (8 \text{ uops} \times 4) \times 10^9 + 4+4) \text{ bytes} = (32 \times 10^9 + 12) \text{ bytes}$$

$$\text{AB-uops} = 32 \times 10^9 \text{ bytes} / 6,4 \text{ s} = 5 \text{ Gbytes/s}$$

(ver Problema 3 del Tema 5)

Una aplicación P que contiene 270×10^{13} instrucciones dinámicas, se ejecuta en un sistema con un procesador CISC y un solo disco (PC1) y se han medido un valor de 1800 MFLOPS y un tiempo de ejecución de 125 horas. Un estudio más detallado sobre el comportamiento de P al ejecutarla en PC1 nos dice que la aplicación está formada por 3 fases:

- La Fase 1 que pasa todo el tiempo leyendo bloques del disco. Esta fase es secuencial y no puede paralelizarse
- La Fase 2 que realiza cálculos que son paralelizables y ocupa el 80% del tiempo de la ejecución de la aplicación P
- La Fase 3 que pasa todo el tiempo escribiendo bloques al disco. Esta fase es secuencial y no puede paralelizarse

Con el objetivo de reducir el coste en tiempo de P lo ejecutamos en un multiprocesador (PC2) con 5 procesadores idénticos al CISC y con un RAID 0 formado por 5 discos también idénticos al del PC1. Sabemos que el tiempo de cálculo de la Fase 2 se puede dividir de forma perfecta entre los procesadores que se usan pero existe un tiempo de sincronización entre los distintos procesos el cual aumenta en un 0,8% del tiempo de ejecución en el sistema PC1 por cada procesador. Al pasar el programa al multiprocesador las instrucciones dinámicas de sincronización han resultado ser de 9×10^{13} en total. Sabemos también que en la aplicación P, el tiempo donde se trabaja con los discos está igualmente repartido entre lecturas y escrituras, que siempre hay suficientes accesos como para saturar el ancho de banda de los discos y que todos los accesos son aleatorios.

- d) **Calcula** el tiempo de ejecución total, los MFLOPS y MIPS al ejecutar la aplicación P en el multiprocesador PC2.

$$\text{Texe}_{\text{PC2}} = (0,2 \times 125) / 5 \text{ h/es} + (0,8 \times 125) / 5 \text{ h/ca} + 0,008 \times 125 \times 5 \text{ h/sinc} = (5 + 20 + 5) \text{ h} = 30 \text{ h}$$

$$\text{MFLOPS} = \text{MFLOPS}_{\text{PC1}} * \text{speed-up} = 1800 \text{ MFLOPS} * (125/30) = 7500 \text{ MFLOPS}$$

$$\text{MIPS} = (270 \times 10^{13} \text{ inst} + 9 \times 10^{13} \text{ inst}) / (30 \text{ h} * 3600 \text{ s/h} * 10^6 \text{ ins/Mins}) = 25833 \text{ MIPS}$$

(ver Problema 3 del Tema 6)

La potencia consumida por cualquier procesador es de 70 W, mientras que la potencia consumida por cada disco (todos los discos son iguales) es de 30 W. Suponemos que estos son los únicos elementos con consumo significativo y que se encuentran funcionando durante toda la ejecución de la aplicación P.

- e) **Calcula** los MFLOPS/W tanto del PC1 (1 procesador + 1 disco) como del PC2 (5 procesadores + 5 discos).

$$\text{MFLOPS/W PC1} = 1800 \text{ MFLOPS} / (70 + 30) \text{ W} = 18 \text{ MFLOPS/W}$$

$$\text{MFLOPS/W PC2} = 7500 \text{ MFLOPS} / (5 \text{ proc} * 70 + 5 \text{ d} * 30) \text{ W} = 15 \text{ MFLOPS/W}$$

(ver Problema 3 del Tema 6)

- f) **Calcula** el incremento de eficiencia energética que podríamos obtener para el multiprocesador si consiguiéramos apagar completamente los elementos que no se utilizan (4 de los 5 procesadores durante las fases secuenciales y los 5 discos durante la etapa de cálculo).

La (Fase 1 + Fase 3) tarda 5 h y usa 1 procesador + 5 discos

La Fase 2 tarda 25 h y usa 5 procesadores + 0 discos

$$\text{MFLOPS/W} = 7500 \text{ Mflops} / ((5 \text{ h} * (1 \text{ p} * 70 \text{ w} + 5 \text{ d} * 30 \text{ w}) + 25 \text{ h} * 5 \text{ p} * 70 \text{ w}) / 30 \text{ h}) = 22,8 \text{ MFLOPS/W}$$

$$\text{MFLOPS/W} = 7500 / (5 * 70 / 30 + 5 * 5 * 30 / 30 + 25 * 5 * 70 / 30) = 22,8 \text{ MFLOPS/W}$$

$$G = 22,8 / 15 = 1,52$$

(ver Problema 3 del Tema 6)